



INTEGRATION REPORT · MAZJ TO REKAZ

Rekaz Merchant Public API: defect and gap report

15 findings from building a production booking integration. Most were measured by calling the live API with our own merchant credentials, and every such item says so on its Evidence line. Where a finding rests instead on the documented surface, on the dashboard, or on a question we asked in July 2026 and never had answered, the Evidence line says that instead. We have not asserted behaviour we did not observe.

Each finding carries an **expected behaviour** and an **acceptance criterion**, so any item can be picked up and closed without coming back to us for clarification.

Prepared by
MAZJ, Al Khobar
info@mazj.org

Date
28 July 2026

Tenant at time of testing
562 reservations, 101 subscriptions,
287 customers, 4 products, 1 branch

What this is, and why we wrote it

We built booking directly into our own website against this API rather than sending customers to the Rekaz storefront. That is a vote of confidence: the catalog, availability and reservation model is good enough to build on, and we would rather extend it than replace it.

Getting there took considerably longer than it should have. Several of the items below cost real debugging time, and three would have reached production as customer-visible failures had the documentation been trusted. We are sending this because other merchants will hit the same ones, and because most are cheap to fix.

Every item here is one a merchant cannot close on their own. The original review ran to 54 findings. Thirty-nine of them have since been absorbed on our side, at our own cost, and they are deliberately not in this document: a defect we can work around is not something to ask you for. What is left is the fifteen where working around it is not available, or where the workaround leaves the risk exactly where it was.

Most of these are not requests to build something. Before writing this version we read the Rekaz platform's own published front-end resources, and found that the capability behind most of these findings already exists in your product: phone verification on the hosted checkout, coupons, custom domains, refunds with provider reversal, invoices and credit notes, ZATCA submission, cart locks and abandoned carts, several named API keys. In almost every case the gap is not the feature. It is that a merchant integrating over the Merchant Public API cannot reach it. Where that is what we found, the finding says so and quotes the strings we read, so the ask is scoped to exposing something rather than creating it.

Each finding is written to be actionable without a conversation: what the documentation says, what the API does, how that was established, what it costs a merchant, and a test that proves it fixed. Where a claim rests on an inference rather than a measurement, it says so.

How to read this. Findings are grouped by kind and each carries a stable id (REK-046 and so on) that we will use in any follow-up. The numbering is not contiguous, because it is carried over unchanged from the 54-finding review so that any item can be cross-referenced with the earlier correspondence. The two fields that matter most are **Expected behaviour** and **Acceptance criteria**: together they define done, precisely enough to be implemented and verified without a conversation.

SEVERITY	COUNT	MEANING
BLOCKER	5	Blocks a first-party booking experience, or loses money or data silently.
HIGH	4	Causes incorrect behaviour or a real security or compliance exposure.
MEDIUM	6	Costs integration time or forces a permanent workaround.
LOW	0	Worth fixing, low urgency.

If only four things change

1. **REK-046, expose the phone verification you already run.** A returning customer can only be booked by `customerId`, and nothing on the API can establish that whoever typed a mobile number holds it. Your own checkout can: `Settings:Checkout:SendOtpToCustomerBeforeCheckout` is a merchant toggle, and the purpose register behind it already spans checkout, login, attendance and account reset. The same customer is verified booking through your storefront and unverified booking through your API.

2. **REK-026, expose the external payment your dashboard already records.** The single highest-value change available. It would let a merchant collect on their own domain through a Saudi gateway while Rekaz stays the invoicing and ZATCA authority. It also closes REK-025 at the same time.
3. **REK-005, sign the webhook deliveries you already send.** Cheap, standard, and the only one of these four a merchant cannot work around at all. Your own settings expose a webhook URL and describe the retry policy; nothing lets a receiver tell your delivery from anyone else's. For a venue that grants entry on confirmation, that is a stranger in the building.
4. **REK-030, a sandbox.** Every measurement in this report was taken against live customer data, because there is nowhere else to take it. That is also why several findings here carry a caveat rather than a measurement: establishing them would have meant leaving real bookings, or a real outage, on a production tenant.



Identity, and who a booking belongs to

The platform already verifies a customer's phone number on its own checkout. Neither that, nor any way to verify a webhook came from Rekaz, reaches the API.

A returning customer can only be booked by customerId, and the phone verification that would make that binding safe is available on Rekaz's own checkout but not through the API

DOCUMENTED	The documentation distinguishes the two payload shapes (<code>customerId</code> for an existing customer, <code>customerDetails</code> for a new one) but nowhere states what happens when <code>customerDetails</code> carries a mobile number that already exists. Nothing warns that the answer is a hard refusal rather than a merge, and nothing tells a public booking form how to know which case it is in before it writes. The documentation also does not mention that a merchant setting of the same subject governs mobile-number uniqueness elsewhere in the platform, nor that Rekaz operates a customer OTP service that its own storefront checkout can be set to require.
ACTUAL	The two shapes are not interchangeable. <code>customerDetails</code> whose <code>mobileNumber</code> already belongs to a customer is rejected with HTTP 403 and the message <code>Mobile number already exists with different customer</code> (Arabic under <code>Accept-Language: ar</code>). An unknown mobile is accepted and creates the customer. <code>customerId</code> is accepted. So for a customer Rekaz already knows, <code>customerId</code> is the only payload the API will take, and a typed mobile number is by itself sufficient to obtain that <code>customerId</code>: <code>GET /customers?mobileNumber=</code> resolves a mobile number to a single customer record, with or without the leading plus. Both halves of that constraint have a counterpart inside the platform. The refusal message is the platform's own localised customer-domain string, <code>Customer:MobileNumberAlreadyExists</code> = "Mobile number already exists with different customer", and a merchant setting on exactly that subject exists: <code>Settings:Customer:EnsureUniqueMobileNumberForCustomer</code> = "Ensure unique mobile number for customers" ("منع تكرار رقم الجوال"). ⚠ Whether the API write path is governed by that setting is an inference from the shared message, not something establishable from outside; if the API enforces uniqueness unconditionally and independently of it, saying so answers part of this finding. The verification that would make the binding safe is likewise a merchant setting, <code>Settings:Checkout:SendOtpToCustomerBeforeCheckout</code> = "Send OTP to customer before checkout" ("تفعيل تأكيد رقم جوال العميل برمز التحقق قبل الحجز"), and Rekaz operates the service behind it: sending, verifying, code expiry, captcha and blocked-number handling all exist, under a purpose register that already spans checkout confirmation, customer login, attendance checks, account reset and two product-specific logins. No way to reach any of it from the Merchant Public API was found: no OTP send or verify endpoint on the documented surface or on the live one, and no documented field on either write endpoint that carries proof of control.
EVIDENCE	Measured 2026-07-28 on both write endpoints against the live tenant, with the same credential and headers, building the payload up one field at a time. The 403 appears exactly when <code>customerDetails.mobileNumber</code> is added and that number already exists. Rekaz neither deduplicates onto the existing record nor creates a second one: it refuses the write. The lookup that completes the path was measured on the same tenant on the same day: <code>GET /customers?mobileNumber=</code> narrowed 287 customer records to 1, with and without the leading plus, so a mobile number typed into a public form is on its own enough to obtain the <code>customerId</code> the write endpoints then require. ⚠ How many of the tenant's existing records the rule affects was not enumerated. The rule itself is what was measured, and it applies to any mobile number Rekaz already holds. That Rekaz operates phone verification is read from Rekaz's own platform UI string table on 2026-07-29, not inferred. The merchant toggle is <code>Settings:Checkout:SendOtpToCustomerBeforeCheckout</code> = "Send OTP to customer before checkout". It cannot always be switched off: <code>Settings:Otp.HasPackages</code> = "you can't disable otp if you have package services", so Rekaz already treats verified phone ownership as mandatory once a merchant sells multi-use packages. The service behind the toggle is described across the same table: <code>Cart:OtpIsRequired</code> = "OTP is required" ("التحقق من رقم الجوال مطلوب"), <code>SP:OtpCodeMismatch</code> = "Invalid OTP code", <code>SP:OtpCodeIsExpired</code> = "OTP code has expired", <code>Otp:Notification.title</code> = "One Time Password Code:", <code>Otp:TurnstileVerificationFailed</code> = "Captcha verification failed. Please try again.", <code>Otp:BlockedCustomerMobileNumber</code>, and a purpose register of which six values were found:

`Enum:OtpPurpose.ConfirmCheckout` = "Confirming Checkout", `.CustomerLogin` = "Login", `.CheckCustomerAttendanceStatus` = "Checking Status", `.ResetAccount` = "Reset Account", `.BeautyPassLogin` = "Beauty Pass login" and `.Grader` = "Read Report". Four of those six are not checkout, which is why adding one more purpose reads as a small change rather than a new subsystem. ⚠️ Settings of the corresponding names (`Platform.Checkout.SendOtpToCustomerBeforeCheckout`, `Platform.Customer.EnsureUniqueMobileNumberForCustomer`) also appear in a platform configuration payload we retrieved without authenticating, which reports `currentTenant` as null. That payload shows the settings exist and are carried at tenant scope; it is not our tenant's configuration and we do not present it as such.

Rekaz also already reconciles duplicate identities after the fact, under a dedicated staff permission: `Permission:MergeCustomers` = "Merge Customers", with `Customer:MergeInfo:Details` = "All subscriptions, reservations, deals, invoices, tasks, notes, and attachments from the selected customers will be transferred to the main customer. The merged customers will then be deleted." And Rekaz's own bulk customer import meets the same uniqueness rule, answering it with an instruction to a human: `CustomerImport:MobileAlreadyExists` = "This mobile number belongs to an existing customer. Search for it in Customers; remove this row if it is the same customer, or correct the number if it is different."

⚠️ One limit on the above. The claim that no API-reachable equivalent exists rests on a full review of the documented and live endpoint surface on 2026-07-28, plus the absence of any API-side OTP string in the platform table. Absence in a string table is not proof, so that half is stated as what we could not find rather than as a certainty. If such an endpoint already exists, naming it resolves most of this finding immediately.

IMPACT

Two constraints compose into an exposure no merchant on this API can opt out of, and the sharp part is that Rekaz has already solved one of them on its own storefront. A public booking form built on the API collects a mobile number it has no way to verify. The same booking placed through Rekaz's own checkout can be verified, because the merchant can require an OTP before checkout. So the same customer, buying the same product from the same merchant, is verified when they book through Rekaz's storefront and unverified when they book through Rekaz's API. If that number is already known to Rekaz, the only payload the API accepts is `customerId`, which binds the booking to that customer's account. So anyone who knows another person's mobile number can file real bookings against that person's account, and whatever Rekaz raises against those bookings is raised against that account too. ⚠️ Exactly what that is cannot be shown from outside: `invoiceId` is documented on the creation response and did not appear in it, and no endpoint retrieves an invoice (REK-034). A merchant cannot avoid the binding by submitting the details the visitor actually typed, because that path returns 403 and the booking simply fails. The only choice the API leaves is between a booking flow that is broken for every returning customer and one that asserts an identity nobody proved. Rekaz's own bulk customer import meets the identical rule and resolves it the way only a person can:

`CustomerImport:MobileAlreadyExists` tells the operator to search the customer list and decide whether it is the same person. A staff member importing a spreadsheet can go and look. A public booking form has nobody to ask, and the API gives it no way to tell the two cases apart before it writes. Rekaz's own product also recognises what is at stake: `Settings:Otp.HasPackages` forbids disabling verification for merchants who sell packages, which is precisely the shape of product where an unverified binding draws down someone else's balance. For a merchant that grants building access, issues a credential, or releases a room on a confirmed booking, that same binding also decides who is entitled to walk in.

EXPECTED BEHAVIOUR

Three changes. The first two are the substance, and neither requires a capability Rekaz does not already run.

(1) **Expose the OTP service you already operate to API bookings.** Rekaz already sends, verifies, expires and rate-limits customer OTPs for storefront checkout, behind a purpose register that already spans checkout confirmation, login, attendance checks and account reset. Add a purpose for

merchant-API bookings and two endpoints against it (for example `POST /customers/verification` and `POST /customers/verification/confirm`) returning a short-lived proof token that `POST /reservations/bulk` and `POST /subscriptions` accept. Record the booking as verified and expose that state on the reservation, so a verified booking can be told from an asserted one afterwards. Honour the merchant's existing checkout toggle, including the case where `Settings:Otp.HasPackages` makes verification mandatory, so a merchant does not have to configure the same policy twice in two places.

(2) Accept `customerDetails` for a mobile number that already exists: attach the booking to the existing customer and leave that customer's stored name and email unmodified, so a merchant can submit what the visitor typed without asserting an identity. If that behaviour is meant to follow the merchant's `EnsureUniqueMobileNumberForCustomer` setting, say so in the documentation and make the setting readable from the API. If it does not follow it, say that too. Today an integrator cannot tell whether the 403 is a platform law or a configuration choice, and that uncertainty is most of why it reads as unfixable.

(3) Separately, and independently of both: **403 is the wrong status here**. This is a business-rule conflict, not an authorization failure, and the platform already files it as one internally, since the message is the localised customer-domain string `Customer:MobileNumberAlreadyExists`. It arrives with the same status as a genuine credential refusal and as the edge's own block, so an integrator debugs credentials for what is a data problem. It should be **409 Conflict** carrying a stable machine-readable code, which does not exist today: `error.code` was null in every legacy-envelope error response observed, leaving a localised human message as the only thing a client could branch on.

Either a booking submitted with `customerDetails` for an already-registered mobile number succeeds and leaves the stored customer record unmodified, or the OTP flow Rekaz already operates for its own checkout is reachable from the API: a code is sent to the number, confirmed, and the resulting proof is accepted on `POST /reservations/bulk` and `POST /subscriptions`, while a proof issued for a different number or an expired one is rejected with a documented 4xx code rather than silently ignored, and the reservation exposes whether it was verified. In both cases, any refusal that remains returns 409 with a non-null documented `error.code`, never 403.

ACCEPTANCE CRITERIA



Webhook deliveries cannot be verified: no signature is documented, and there is nowhere a merchant can read or set a signing secret

DOCUMENTED	21 events across four groups, delivered with 10 retries and exponential backoff. No signing mechanism is documented.
ACTUAL	No HMAC, no shared secret, no signing header and no timestamp binding appears in the documented envelope or anywhere on the configuration surface, so a receiver has nothing to verify a delivery against beyond the secrecy of the URL itself. The merchant-facing configuration is a URL and an on/off switch: in the platform configuration payload the outbound-webhook entries are <code>Platform.OutboundWebhook.Enabled</code>, <code>Platform.OutboundWebhook.Url</code> and a legacy <code>Platform.Webhook.Url</code>, with no secret beside them, and the screen's own strings cover the toggle, the URL, an <code>https://</code> validation message and an explanation of delivery retries. ⚠️ Stated precisely, because it is the difference between a finding and a guess: the documented envelope, the registration screen and Rekaz's own English and Arabic UI strings were all searched, and no signing mechanism of any kind appears in them. A signature computed server-side with no merchant-facing surface would leave no trace in any of those, so this is what could not be found rather than a proof of absence. What it does establish is that a merchant has nowhere to read or set a secret, which is what a receiver needs before it can verify anything, and no per-event subscription and no test delivery either.
EVIDENCE	Inspected the webhook registration surface at <code>platform.rekaz.io/Identity/apikeys</code> and the documented payload envelope, 2026-07-28. The registration screen offers no signing secret and no signature option. Checked against Rekaz's own platform UI string table and configuration payload, 2026-07-29. Seven outbound-webhook strings were found, and none of them concerns authenticity: <code>Settings:OutboundWebhook:Enabled</code> = "Enable Webhooks", <code>Settings:OutboundWebhook:Url</code> = "Webhook Url", <code>OutboundWebhook:Settings</code> = "Webhook Settings", <code>OutboundWebhook:Enabled</code> = "Enable Webhook", <code>OutboundWebhook:Url</code> = "Webhook Url" (Arabic "الذي سيتم إرسال Webhook رابط الـ" / "البيانات إليه"), <code>OutboundWebhook:Url:Validation</code> = "The Webhook URL must start with 'https://.'" and <code>OutboundWebhook:Explanation</code>. The last is by far the longest and is entirely about delivery reliability: in Rekaz's own words "the system will automatically try again multiple times", and then "up to 10 attempts total. These attempts occur over time, providing multiple opportunities for temporary problems to be resolved. If all 10 attempts fail, the system will stop trying and will log the failure.", after which the string points merchants to docs.rekaz.io. Nothing in it concerns who sent the delivery. Searching the English and Arabic string tables for signature, signing, HMAC, secret, shared secret, token, callback, nonce, timestamp, integrity, tamper, replay, allowlist and test delivery, and for توقيع, تشفير, المفتاح السري, ويب هوك, returned no webhook-related hit in either language; the same search across the hosted checkout bundle returned nothing; and the configuration payload carries no webhook secret among its <code>Platform.*</code> entries. ⚠️ A string table proves presence, not absence. ⚠️ That payload was also served without authentication and reports <code>currentTenant</code> as null, so it is a platform or default payload rather than any one merchant's configuration. ⚠️ No webhook delivery has been received or inspected. This finding rests on the absence of any documented or configurable signing mechanism, not on an inspected delivery.
IMPACT	Anyone who learns or guesses a merchant's webhook URL can forge a <code>ReservationConfirmedEvent</code> and tell that merchant's systems that an unpaid booking is paid. For a merchant that grants building access, issues a QR credential, or releases a room on confirmation, that is a physical or commercial grant made on an unauthenticated HTTP request from an unknown sender. The only defence available without a signature is for the receiver to treat every delivery as an untrusted hint and re-fetch the entity by id before acting, which removes most of the value of webhooks and adds a call to an API whose read endpoints were measured between 0.8s and 10.8s, with the reservation list itself at around 6s per page (REK-042). It is also unauditably after the fact: an unsigned payload leaves no evidence of who sent it. ⚠️ Rekaz applies a different standard where it is the receiver. Its own integration settings hold a Tamara notification token (<code>Integration:Tamara:NotificationToken</code> = "Notification Token", Arabic "مفتاح التنبيه" / "Notification Token") and a Tabby secret key (<code>Integration:Tabby:SecretKey</code> = "Secret Key", Arabic "مفتاح الربط السري" / "Secret Key"). The platform holds shared secrets with its own payment

vendors and issues none to its merchants. ⚠️ There is also a second place where Rekaz posts event data to a merchant-supplied endpoint with nothing attached to authenticate it: the WhatsApp workflow builder offers `WhatsappWorkflows:SectionTwo:OptionEight` = "Send event data to a webhook" ("إرسال بيانات الحدث إلى خطاف ويب") as an action, configured by `Webhook:PostUrl` = "Post Url", on triggers that include "A new cart order is received". A URL and no secret, again, which makes this a pattern rather than an oversight on one screen. ⚠️ And it is not a free convenience: webhooks are sold. `Addon:ThirdPartyApi:Upsell:Benefit2` = "Webhook integrations" ("تكاملات Webhook") sits under `Addon:ThirdPartyApi:Upsell:Title` = "API Access - Premium Addon", so this is a paid feature shipping without message authentication.

**EXPECTED
BEHAVIOUR**

Sign the raw request body with HMAC-SHA256 keyed on a per-merchant secret that is visible in the dashboard and independently rotatable. Bind a timestamp inside the signed material to prevent replay, and send signature and timestamp as headers. This is the standard pattern across Stripe, Tap, Moyasar and Paylink, so integrators already know how to consume it.

**ACCEPTANCE
CRITERIA**

Every webhook delivery carries a signature header. A merchant can recompute the signature from the raw body plus their secret, and can reject a payload whose signature does not match or whose timestamp falls outside an acceptable window. The verification procedure is documented with a worked example, and the signing secret can be rotated without dropping deliveries.



The payment boundary

Every item here exists inside Rekaz already. What is missing is a merchant's ability to reach it programmatically.

The Merchant Public API offers no payment path, although the Rekaz-hosted checkout already runs one

DOCUMENTED

No payments API is offered anywhere in the documentation.

ACTUAL

The only path to payment offered to an API integrator is the Rekaz-hosted checkout, reached through the `paymentLink` returned by a write. Nothing in the documentation, and nothing on the endpoint surface reachable with a merchant credential, accepts card data or a gateway token, issues a payment intent, or exposes an embeddable payment element. ⚠️ The architecture this finding asks for is not new work. Rekaz's own hosted checkout already runs it on its Moyasar branch: the checkout renders its own card inputs, tokenises them in the browser through a Moyasar helper called with a publishable key delivered in the page's settings payload, and posts only the resulting token to a Rekaz call, together with the order's `encryptedAmount`. Which gateway collects is a per-tenant setting and the checkout carries three branches, `moyasar`, `myfatoorah` and `rekaz-pay`. The other two work differently, embedding MyFatoorah's own card widget and exchanging a `sessionId` rather than a token, so the tokenisation flow described here is the Moyasar branch specifically. On that branch the pattern is already in production. It is simply not reachable with an API key.

EVIDENCE

Full review of the documented and live endpoint surface against the production tenant, 2026-07-28. The `paymentLink` behaviour described under Impact was measured separately, on a real reservation created against the live tenant on 2026-07-27. The documentation shows an absolute URL, `{"paymentLink": "https://platform.rekaz.io/i/NcRo"}`. The live API returned `{"paymentLink": "/orders/pay/RMogH0PQc47FStqK"}`, a relative path in a different path shape, with nothing in the response identifying the host it is meant to be resolved against. The same path was then resolved against both plausible hosts: `platform.rekaz.io/orders/pay/<id>` answers 200, while the same path on the merchant's own store domain answers 404, so the correct host cannot be inferred. ⚠️ The same `paymentLink` field is documented on `POST /subscriptions`, but no subscription was created, so that endpoint was not separately verified. The hosted-checkout behaviour described under Actual is read from Rekaz's own shipped checkout bundle, not measured. Its gateway switch reads `settingsData.paymentGateway` and branches three ways, on `moyasar`, on `myfatoorah` or `rekaz-pay`, and on anything else it logs `[CardPaymentForm] Unknown gateway, returning null`. On the Moyasar branch the bundle collects the card in its own inputs with its own validation, tokenises through a helper imported from a Moyasar API chunk and called with those fields and `settingsData.moyasarPublishableKey`, then posts the result in a call shaped `{token, tokenType: "token", encryptedAmount, callbackSource, callbackReference}`; the Apple Pay path on the same branch uses the same call with `tokenType: "applepay"`. On the MyFatoorah and Rekaz Pay branches the bundle instead loads a script from `settingsData.myFatoorahScriptUrl`, renders MyFatoorah's own card widget into a container, passes `CustomerPayment:CardholderName`, `CustomerPayment:CardNumber`, `CustomerPayment:ExpiryDate` and `CustomerPayment:CVV` into it as label text, and exchanges a `sessionId` rather than a token. Which of the three this merchant's tenant runs has not been established from outside, so nothing here should be read as a description of this tenant. ⚠️ These are observations of client code. They establish that the pattern exists in the product; they do not establish an API contract, a plan gate, or whose credential the publishable key belongs to. On that last point, the unauthenticated platform configuration, served with no tenant resolved (its `currentTenant` block carries a null id and a null name and is marked not available), carries `"Platform.PaymentGateway.Provider": "moyasar"` and a `"Platform.PaymentGateway.Moyasar.PublishableKey"` whose value begins `pk_live_`, which reads as a platform-level setting rather than a per-merchant one. We also searched both platform string tables for a public tokenisation endpoint, a payment intent, an embeddable element or an iframe and found nothing, which is inconclusive: a server-side capability need not have a UI string.

IMPACT

This is the one hard architectural constraint on building a first-party booking experience. A merchant can own the catalog, availability, the booking form and the confirmation, and still has to hand the buyer to a page the merchant does not serve for the step that takes the money, which is the step merchants most commonly cite as their abandonment point. It also raises the cost of every other defect at that

boundary: the link to that page comes back as a relative path (/orders/pay/<id>) where the documentation shows an absolute URL, so a merchant who follows the documentation resolves it against their own origin and lands the buyer on a 404; that link is pinned to a Rekaz host rather than the merchant's own linked store domain, and the Apple Pay sheet served on it names the payee Rekaz rather than the merchant (REK-027); and it never returns the buyer (REK-049). The one step a merchant cannot control is the step with the most defects in it. ⚠️ The cost of closing this is smaller than the cost of leaving it open, because the mechanism is already written and running on at least one gateway branch. What is missing is the entry point.

**EXPECTED
BEHAVIOUR**

Two acceptable shapes, and the cheaper one is not this finding. (1) Expose the payment path the hosted checkout already runs: a client key scoped to a single order, plus an authenticated endpoint that accepts a gateway token against that order together with the order's encryptedAmount , in the same shape as the tokenType: "token" call the checkout makes today. A key scoped to one order is what makes this safe to issue to an integration, and is the reason this finding does not ask for a gateway account credential of any kind. (2) Ship REK-026, which is cheaper and which satisfies the acceptance below. These two findings are one remediation item and not two, and REK-026 is where it should be tracked.

**ACCEPTANCE
CRITERIA**

A merchant can take payment for a Rekaz order without the buyer leaving the merchant's own domain, and Rekaz then reports the order as paid, issues its invoice, moves the reservation or subscription out of Pending , and fires the same webhook events as a Rekaz-collected payment.

The dashboard can mark an invoice paid from money collected outside Rekaz; the API cannot

DOCUMENTED	Not offered on the Merchant Public API. No documented endpoint records a payment, accepts an external payment reference, or confirms a bank transfer.
ACTUAL	No endpoint on the Merchant Public API accepts a payment record, an external payment reference, a payment method or a paid timestamp, so an integration has no way to tell Rekaz that money was collected outside Rekaz's own online rails. <code>PUT /reservations/{id}/confirm</code> does exist on the endpoint surface, but the documentation does not state whether it settles the order, issues the invoice, or only moves the reservation's status, and nothing documented on it carries a payment reference, an amount, a method or a date. 🚫 This is a parity gap, not a missing feature. The Rekaz dashboard already performs this operation. A merchant can define their own payment methods for internal collection, record a payment against an invoice with an amount, a method and a payment date, and confirm a customer's bank transfer into the merchant's own bank account, which Rekaz's own copy describes as separate from the account Rekaz settles earnings into, so that the invoice flips to paid. The reservation status transition this finding asks for is already wired to that event, and in the platform configuration it is on by default. What is missing is the API entry point, not the behaviour.
EVIDENCE	Raised as question 2 in our July 2026 review letter and never answered. Re-confirmed absent from the documented and live Merchant Public API surface on 2026-07-28. 🚩 No payment has been completed against this tenant through the API, and <code>PUT /reservations/{id}/confirm</code> has not been called, so what either of those does to an order is stated here only as far as the documentation states it. The dashboard capability described under Actual is read from Rekaz's own shipped platform strings. <code>Invoice:ConfirmBankTransfer:Hint</code> = "A bank transfer is waiting for your confirmation. Once you've verified the amount reached your account, confirm it to mark the invoice as paid."; <code>Invoice:ConfirmBankTransfer:Confirmation</code> = "Have you verified that the bank transfer amount has arrived in your account?"; <code>BankAccount:Warning:AccountSeparation</code> = "This account is yours and separate from the bank account where Rekaz deposits your earnings."; <code>BankAccount:Warning:TransferConfirmation</code> = "Confirming receipt of the transfer is handled by you directly."; <code>Invoice:PaymentMethods:Help</code> = "Define the payment methods available when paying invoices internally."; <code>Invoice:PaymentMethodName:Placeholder</code> = "e.g. Local Bank Transfer"; <code>Invoice:ConfirmPayment</code>, <code>Invoice:PaymentDate</code> and <code>Invoice:CollectedAt</code>; <code>POS:Payment:CustomMethods</code> = "Custom payment methods"; <code>POS:PaymentEditor:PartialPaymentRecorded</code> = "Partial payment recorded"; the <code>Enum:OrderPaymentMethod</code> set, which includes <code>.Cash</code>, <code>.BankTransfer</code>, <code>.PointOfSales</code>, <code>.Gift</code> and <code>.Other</code>; <code>Enum:OnlinePaymentMethod.UnknownInReport</code> = "Manually Inserted"; and <code>SP:TransactionNotModifiable</code> = "This transaction was processed by a payment gateway and cannot be modified or deleted", which implies that transactions not processed by a gateway are merchant-created and editable. The status transition is <code>Settings:Reservation:ConfirmReservationAutomaticallyIfInvoicePaid</code> = "Confirm reservation automatically if invoice is paid", and the unauthenticated platform configuration carries <code>"Platform.Reservation.ConfirmReservationAutomaticallyIfInvoicePaid":"True"</code>. That same payload carries <code>"Platform.OutboundWebhook.Enabled"</code>, <code>"Platform.OutboundWebhook.Url"</code> and <code>"Platform.Webhook.Url"</code>, which is what the acceptance below assumes when it asks for a webhook delivery. 🚩 These are platform strings and platform-level settings, read with no tenant resolved. They establish that the capability exists in the product. They do not establish which plan reaches it, how this merchant's tenant is configured, or an API contract. We searched both string tables for any public API endpoint that records a payment, for an external payment reference and for an idempotency key, and found nothing, which is inconclusive for the same reason.
IMPACT	🚫 This is the highest-value single change in this report, and it is a small one, because Rekaz already performs the operation: only the API entry point is missing. Without it, an integration that collects payment itself has to keep two disconnected records: the money in the merchant's own gateway, and an order in Rekaz that Rekaz still believes is unpaid. Everything Rekaz is relied on for downstream then

drifts: the invoice, the reservation status, the sales and collection reports, and in Saudi Arabia the ZATCA e-invoicing obligation, which lands on whoever is the invoicing system of record. What Rekaz files for ZATCA cannot be confirmed from the API today, because no endpoint retrieves an invoice (REK-034). The alternative is to give up owning the payment step, with everything that follows from it (REK-025, REK-027, REK-049). With this endpoint a merchant keeps Rekaz as the system of record for the catalog, the booking and the invoice, and owns the payment step, which is the division of responsibility most merchants of this size want. ⚠️ Two things this finding is deliberately not asking for, because Rekaz already ships both. First, bringing a merchant's own gateway account to Rekaz:

`BusinessSettings:PaymentGateway` = "Payment Gateway" exists, and `KYC:MyFatoorahNote:Message` = "To activate online payments, please register with MyFatoorah to get your API key, then contact technical support to complete the activation." That routes the money through a chosen gateway, but the buyer still pays on a Rekaz surface. Second, offering bank transfer or payment on arrival at checkout: `RekazPay:PaymentMethod:BankTransfer:EnableMessage` = "Customers can choose bank transfer and use your saved bank account details to pay." and

`RekazPay:PaymentMethod:CashOnDelivery:EnableMessage` = "Customers can place the order now and pay when they arrive at your branch." Those two also take money that never touches Rekaz's online rails, which is precisely why the recording behaviour already exists, but the buyer still selects them on a Rekaz surface and the merchant still reconciles them by hand in the dashboard. Neither lets a payment captured on the merchant's own domain, in the merchant's own gateway session, be posted to a Rekaz order by an integration. That is the only thing this finding asks for.

EXPECTED BEHAVIOUR

Expose the payment-recording operation the dashboard already performs. An authenticated endpoint on the order, for example `POST /orders/{id}/payments`, accepting an amount, a currency, a payment method drawn from the same set the dashboard offers (`Cash`, `BankTransfer`, `PointOfSales`, `Other`, plus the merchant's own custom methods), a payment date and an external payment reference. It marks the order paid, issues the Rekaz invoice, and fires the same webhook events as a Rekaz-collected payment, with the same effect on reservation status that `Settings:Reservation:ConfirmReservationAutomaticallyIfInvoicePaid` already produces, and the documentation states how it relates to the existing confirm endpoint. An amount that does not match the order is rejected with a documented error code rather than accepted. The external reference is treated as an idempotency key, so a retried confirmation cannot double-pay an order.

ACCEPTANCE CRITERIA

A merchant collects payment through their own gateway, calls the endpoint once with the gateway reference, and can then observe all four of: the reservation reads `Confirmed` on `GET /reservations/{id}`; its `orderPaymentStatusString` reports paid; a delivery carrying the same event name as a Rekaz-collected payment arrives at the merchant's registered webhook URL; and the payment appears in the dashboard's invoice transactions exactly as a dashboard-recorded payment does. A second call with the same external reference changes nothing and returns the original response. Where the invoice retrieval requested in REK-034 has also shipped, the invoice returned for that order carries the external reference; until it has, that condition is not part of this finding's acceptance.

The payment link ignores the merchant's own linked domain, and the Apple Pay sheet names Rekaz as the payee

DOCUMENTED	Not addressed. The documentation does not state which host <code>paymentLink</code> resolves against, or whether a merchant's linked custom domain can serve it.
ACTUAL	The payment link returned on a real reservation created on this tenant resolves against <code>platform.rekaz.io</code> . It does not resolve against this merchant's own store domain, which is linked through Rekaz and serving this merchant's Rekaz storefront today. Separately, on the hosted checkout's Apple Pay path, the merchant name sent to the gateway for merchant validation is the literal string <code>Rekaz</code> , for every merchant. Only the sheet's line-item label is populated from the merchant's business name. This is not a missing capability, and the finding is narrower than it may first appear. Rekaz already ships the rest of it. The hosted checkout already renders the merchant's own logo and business name from its settings payload. Custom domains are already a first-class, dashboard-managed, plan-gated feature. Rekaz's own back office already registers a merchant domain with the payment gateways, which is the step that exists so that payment UI can be served from that domain. What is left is that the link the public API hands back uses none of it, and that one hardcoded name on the Apple Pay sheet.
EVIDENCE	Raised as question 1 in our July 2026 review letter and never answered. The payment link returned on a real reservation created 2026-07-27 was the relative path <code>/orders/pay/<id></code>. Resolved against <code>platform.rekaz.io</code> it answers 200. Resolved against this merchant's own store domain, which serves this merchant's Rekaz storefront, it answers 404 after a 308 that adds a locale prefix. Why it 404s there was not established, and only Rekaz can say what the intended behaviour is. The platform capabilities described under Actual are read from Rekaz's own shipped code and strings. The hosted checkout bundle reads <code>settingsData.logoUrl</code> in three separate components and renders it as an image, including on the order summary and on the success screen, and it renders <code>settingsData.businessName</code> as visible text under a <code>CustomerPayment:BusinessName</code> label. <code>Settings:Basic:Logo</code> = "Brand Logo", described by <code>Settings:Basic:Logo:Description</code> = "The logo appears in the website header and invoices". <code>Addon:RemoveMadeByRekaz:Description</code> = "Remove 'Made by Rekaz' branding from all your pages". On the domain feature: <code>Domain:Settings:Subtitle</code> = "Manage your store address, link a custom domain, and control DNS records."; <code>Domain:Choose:LinkDescription</code> = "Own a domain? Link it to your store for free by pointing your nameservers."; and <code>Domain:Choose:GatedNote</code> = "Custom domain linking is available on paid plans.", consistent with the unauthenticated platform configuration, which carries <code>Platform.CustomDomain</code> twice, once as a setting with an empty value and once among its feature flags with the value <code>"false"</code>. That this feature is the store hostname rather than a mail domain is stated by <code>Domain:Delete:Confirmation</code> = "Once deleted, your website will no longer be reachable through this Domain, and all defined DNS records will be permanently lost. This action cannot be undone." In the back office, <code>Domain:VerifyMoyasarConfirmMessage</code> = "Are you sure you want to verify the domain {0} with Moyasar?" and <code>Domain:VerifyMyFatoorahConfirmMessage</code> = "Are you sure you want to verify the domain {0} with MyFatoorah?", over a domains table carrying <code>Domain:Moyasar</code> and <code>Domain:MyFatoorah</code> columns. Consistently with a checkout intended to run on more than one host, its Apple Pay merchant validation sends the live hostname rather than a constant. That is also where the hardcoded name is, in the same object, shown here with the bundle's minified variable names left out: <code>{validation_url, display_name: "Rekaz", domain_name: window.location.hostname, publishable_api_key}</code>. ⚠️ These observations establish that the capabilities exist in the product. They do not establish that they are available on this merchant's plan, and only Rekaz can confirm the intended behaviour.
IMPACT	Every purchase made through the API ends on a host the buyer does not recognise, at the moment the buyer is being asked to enter card details, while the merchant's own domain, already linked through Rekaz and already serving this merchant's Rekaz storefront, sits unused for this purpose. On Apple Pay the mismatch is on the sheet itself: the payee is presented as <code>Rekaz</code> , so a customer is asked to authorise a payment to a company they did not choose to buy from, with the merchant's name

appearing only on the line item below it. A merchant has no answer for a customer who asks whether the payment page is genuine, and no way to teach customers which domain is theirs. Neither half is expensive: one is a link, and one is a string. This is a much smaller change than REK-026 and worth shipping on its own merits even if REK-026 never happens.

**EXPECTED
BEHAVIOUR**

Two changes. First, return `paymentLink` as an absolute URL in every case, and document which host it resolves against. Where a tenant has a linked custom domain that is verified with the payment gateway, serve the hosted checkout from that domain and return that host; where a tenant has none, keep the current host. A dedicated payment hostname, for example a `pay.` subdomain, on the same DNS and TLS machinery the domain feature already uses, would be an acceptable alternative. Second, send the merchant's configured business name as the Apple Pay merchant display name instead of the literal `Rekaz`. If the gateway relationship requires the literal, say so in the documentation, so that a merchant can set the expectation with customers rather than discover it at the payment sheet.

**ACCEPTANCE
CRITERIA**

A merchant with a linked, gateway-verified custom domain creates a booking through the API, and the URL in `paymentLink` is absolute, resolves to the merchant's own hostname over valid TLS, renders the merchant's configured logo and business name, and completes both a card payment and an Apple Pay payment on that hostname, with the Apple Pay sheet naming the merchant rather than `Rekaz`. A merchant with no custom domain receives an absolute URL on the current host, and the documentation states which host applies in each case.

No return URL on the booking API, so a paying customer never comes back, although Rekaz already computes return links internally

DOCUMENTED	No return, callback or redirect parameter is documented on either booking creation call.
ACTUAL	No such field exists in the request to <code>POST /reservations/bulk</code> or <code>POST /subscriptions</code> , and no field in the response describes where the buyer goes after payment. The buyer's journey ends on the Rekaz-hosted page, whose outbound actions are downloading an invoice, a receipt or an order summary, or emailing the invoice to themselves. No link back to a merchant's own site was found anywhere in that page's bundle. ⚠️ The mechanism itself already exists inside Rekaz. The order object the hosted checkout consumes carries a <code>links</code> structure with per-gateway <code>success</code> , <code>cancel</code> and <code>failure</code> paths plus a <code>paymentCallback</code> path, and the settings payload carries a <code>callbackBaseUrl</code> that those paths are resolved against, falling back to the page's own origin. The payment call also carries <code>callbackSource</code> and <code>callbackReference</code> , and a gateway-supplied <code>redirectUrl</code> is honoured when one comes back. So Rekaz already computes a per-order, per-outcome return destination. The public API neither accepts a destination nor returns the ones Rekaz computes.
EVIDENCE	Documented and live surface review of both write endpoints, 2026-07-28. Confirmed against the response of a real reservation created 2026-07-27, which carried <code>paymentLink</code> and nothing describing where the buyer goes afterwards. The internal return-URL machinery described under Actual is read from Rekaz's own hosted checkout bundle: <code>settingsData.callbackBaseUrl</code> falling back to <code>window.location.origin</code>, then <code>success_url</code>, <code>cancel_url</code> and <code>failure_url</code> built from that base plus <code>order.links.tabby.success</code>, <code>.cancel</code> and <code>.failure</code>, with the identical shape for Tamara; plus <code>order.links.paymentCallback</code>, which is resolved against the same base and passed as the callback URL when a MyFatoorah session is initiated; plus the <code>callbackSource</code> and <code>callbackReference</code> parameters on the payment call. Both post-payment paths honour a gateway-supplied <code>redirectUrl</code>, by assigning <code>window.location.href</code> or calling <code>window.location.replace</code>. We then enumerated the outbound links in that bundle: they go to an order summary, an invoice, a receipt, the page's own path, and Rekaz's own plans page, and none goes to a merchant site. <code>CustomerPayment:ReturnToDashboard</code> = "Return to Dashboard" exists in the platform string tables but does not appear in this bundle at all, so it belongs to a merchant-facing surface rather than to the buyer's checkout. Rekaz's own storefront, meanwhile, has the confirmation page this finding asks for: <code>Checkout:SuccessDescription</code> = "Thank you for purchasing from our site, your order {0} is created successfully" and <code>Checkout:GoToHome</code> = "Home Page". ⚠️ We also searched both platform string tables for a merchant-configurable return URL, in English and in Arabic (returnUrl, redirect, callback, success url, cancel url, thank you, back to store, and the Arabic equivalents), and found none. That is not evidence that none exists, and the unauthenticated platform configuration demonstrates why: <code>"Platform.OutboundWebhook.Enabled"</code>, <code>"Platform.OutboundWebhook.Url"</code> and <code>"Platform.Webhook.Url"</code> are real settings for outbound URLs whose entire footprint across both string tables is a single key. <code>Webhook:PostUrl</code> = "Post Url". A server-side setting need not have a UI string, and <code>callbackBaseUrl</code> plainly is one.
IMPACT	A merchant who builds booking on their own domain loses the customer at the last step. There is no confirmation page in the merchant's own brand, no place to show what was booked and what happens next, and no client-side signal that payment completed, so the outcome is known only from a webhook, which carries no signature (REK-005), or by polling the reservation list and matching the booking by hand. For a venue selling access to a physical space, the minutes after payment are when a first-time customer needs directions, entry instructions and a welcome, and the API provides no moment in which to deliver them. Closing this is unusually cheap, because the destination is already computed per order and per outcome: what is missing is a way for a merchant to supply one and a field that returns it.
EXPECTED BEHAVIOUR	Accept <code>returnUrl</code> and <code>cancelUrl</code> on both booking creation calls, validated against a merchant-configured allow-list of origins, and resolve them through the same <code>callbackBaseUrl</code> and <code>order.links</code> mechanism the hosted checkout already uses. After the payment attempt, redirect the buyer there carrying at least the order reference, the reservation number and the outcome as query parameters. Signing those parameters with the same per-merchant secret proposed in REK-005

would let a merchant identify the booking without a second call. If `callbackBaseUrl` is already merchant-configurable, saying so and returning the computed `links` object on the booking response would close most of this finding on its own.

**ACCEPTANCE
CRITERIA**

A booking created with a `returnUrl` returns the buyer to that URL after both a successful and a failed payment attempt, carrying the order reference, the reservation number and the outcome as query parameters, which is enough to identify the booking without a second API call. A `returnUrl` outside the configured allow-list is rejected at creation time with a 4xx naming the field. The booking response states where the buyer will be sent, rather than leaving it implicit.



Operating without information

Behaviour the platform models internally, and manages in its own dashboard, that an API integrator cannot observe, measure or test.

Unpaid items already hold slots inside the platform and carts already expire, but neither reaches the API, so an integrator cannot tell a held slot from a sold one**DOCUMENTED**

Silent. The documentation does not state whether a `Pending` reservation holds its slot, whether an unpaid reservation expires, or after how long. Nothing in the API reference mentions a cart, a cart lock or an expired cart, although the platform has all three.

ACTUAL

In the platform the mechanism exists, and merchants can see it. The reservation calendar renders unpaid items as holds: `Reservation:Calendar:CartLockLabel = Reserved in cart`, beside `Reservation:Calendar:CartLockDuration = Hold for {duration}`. ⚠️ That second key is not read here as proof of a hold expiry window. Its neighbour in the same calendar block, `Reservation:Calendar:BufferDuration = Rest for {duration}`, labels a block with that block's own span, so `Hold for {duration}` may be rendering the length of the held booking rather than the time left before the hold lapses. A string table cannot settle which. Rekaz can settle it in a sentence. What is not ambiguous is that carts reach `Enum:CartStatus:Expired = Expired` by way of `Enum:CartStatus:PendingPayment`, and that the availability engine counts an unpaid item as a blocking cause. The dashboard states that outright: its diagnostics panel (`ReservationsDialog:Diagnostics:Description = This page shows why the reservation slot is unavailable`) names `ReservationsDialog:Diagnostics:Source:pending-cart-item = Pending cart item` as a cause distinct from `ReservationsDialog:Diagnostics:Source:confirmed-reservation = Confirmed booking`, and identifies the blocker through `ReservationsDialog:Diagnostics:CartItemReferenceId = Cart item ID`. A console for the fallout exists too, `Menu:AbandonedCarts`, with its own `Permission:ManageAbandonedCart`. **None of it reaches the API.** `GET /reservations/slots` reports a window as available or not, with no reason and no held-until time, so a slot held by an unpaid item and a slot sold to a paying customer are indistinguishable to an integrator. No documented reservation field carries a hold expiry, and no webhook is documented for a hold being released. ⚠️ The platform's cart, checkout and reservation settings vocabulary was also searched for a merchant-configurable hold length and nothing was found, which is inconclusive rather than proof that no such setting exists. The open question is therefore narrower than this finding originally put it: not whether a hold exists, but whether a reservation created through `POST /reservations/bulk` participates in that cart lock or bypasses it. Rekaz can answer that from its own code. A merchant cannot observe it from outside.

EVIDENCE

I asked this as question 6 in our July 2026 review letter and never received an answer, and it was still undocumented when I re-checked on 2026-07-28. I took the platform strings above from Rekaz's own localization tables, English and Arabic, as loaded by the merchant dashboard, and reviewed them on 2026-07-29; key names and values are reproduced without alteration. The API behaviour is from live measurement against our production tenant on 2026-07-27 and 2026-07-28. `PUT /reservations/{id}/cancel` answered 204 and the cancelled window was available again on the next slots request, but I cannot tell from outside whether that was a hold being released or a slot that was never held in the first place. I was not willing to leave an unpaid booking sitting on a real room for days to find out, and there is nowhere else to try it (REK-030).

IMPACT

Payment always happens off-site (REK-025), so there is always a window between a reservation being created and being paid, and abandoned checkouts are a normal share of any booking funnel. The defect is an asymmetry rather than an absence. Rekaz staff can open the reservation calendar and see which slots are held and by which cart item, and open the diagnostics panel to see why a slot is being refused. Anyone integrating over the API is given none of that, and has to design as though neither the hold nor its expiry exists. That forces a choice between two wrong designs: treat holds as real and build an abandoned-booking cleanup that may duplicate one Rekaz already runs, or treat them as absent and accept that two customers can pay for the same window. Rekaz evidently already sees the second case, because it ships the customer copy for it in `Cart:SlotConflictRefundedSms: We're sorry, the time slot you selected is no longer available. Your payment has been refunded in full. Please book another slot`. A merchant cannot presently tell which of the two failures they are exposed to,

and the cost of guessing wrong is either availability that silently shrinks or a paying customer who has to be refunded and apologised to.

EXPECTED BEHAVIOUR

Document the lifecycle first, using the mechanism that already exists in the platform. State whether an unpaid item holds its slot, whether that hold has a time to live and what it is, and whether a reservation created through `POST /reservations/bulk` takes the same cart lock or bypasses it. If the value rendered by `Reservation:Calendar:CartLockDuration` is the remaining hold, publish it as the time to live; if it is the booked span, say so and publish the time to live separately. Then expose it: return the held-until timestamp on the reservation a write creates, and return the blocking reason on `GET /reservations/slots` using the same source vocabulary the dashboard diagnostics already uses, so that a confirmed booking and a pending cart item are distinguishable to an integrator. Emit a webhook when a hold lapses and the window is released. If the hold length is not already configurable per merchant, make it configurable.

ACCEPTANCE CRITERIA

The documentation states whether a reservation created through the API holds its slot, whether that hold expires and after how long, and names the field carrying the expiry. A booking created and left unpaid returns that field. `GET /reservations/slots` reports the window as unavailable with a machine-readable reason that distinguishes a held slot from a confirmed booking. When the hold lapses, a webhook fires and the window reappears as available in `GET /reservations/slots` within a documented time.

No sandbox is offered to merchants, although Rekaz already maintains seeded test tenants of its own

DOCUMENTED	No sandbox, test tenant or test mode is mentioned anywhere in the documentation.
ACTUAL	None is documented and none was offered when asked. The tenant issued to this merchant is the live one, and it is the only tenant this merchant has been given. Rekaz, however, already maintains seeded test tenants of its own. Its dashboard login page carries a block headed <code>Test accounts</code> (<code>Auth:Login:TestAccounts:Title</code>), sitting between the login error strings and the registration heading and listing ten tenants by name: <code>Rekaz CRM</code> , plus Gym, Salon and Chalet each at Freelancer, Advanced and Premier, which are names of Rekaz's own subscription plans (<code>Plans:Freelancer</code> , <code>Plans:Advanced</code> , <code>Plans:Premier</code>). The seeded catalog data, the per-vertical fixtures and the per-tier feature gating a merchant sandbox would need therefore already exist, and are already being maintained. ⚠️ Two limits on that reading, stated plainly. These are strings from the platform's localization table, which shows the block is built into the dashboard, not that it renders for anyone outside Rekaz. And a dashboard test tenant is not an API sandbox: it implies nothing about separate API credentials, and nothing about a payment step that can be driven to success and to failure without a real card. The dashboard's API-key surface is built around a key name, a Base64 value and a Tenant ID, with rotate and delete; no environment selection appears on it, which is inconclusive rather than proof that none exists. The request below is scoped to the API accordingly.
EVIDENCE	I raised this as question 3 in our July 2026 review letter and never received an answer. Every measurement in this report was consequently taken against our production tenant, which on 2026-07-28 held 562 reservations, 101 subscriptions and 287 customers of real data. I read the test-account strings from Rekaz's own platform localization tables, English and Arabic, as loaded by the merchant dashboard, and reviewed them on 2026-07-29. I also searched both tables for sandbox, test mode, staging, playground, simulate, dry run, test environment, test data, seed data, environment and live mode, and for the Arabic <code>بيئة تجريبية</code> , and found no environment-selection surface. A string table can only show what is present, so that search is inconclusive, and it is not what this finding rests on. What it rests on is that we asked for a sandbox and were not given one. The trial strings those tables do carry belong to Rekaz's own SaaS billing trials, which is a different concept.
IMPACT	Every test a merchant writes runs against real customer records. Write-path tests cannot be automated at all without creating real bookings that operations staff see in the dashboard and that distort reporting, so the part of the integration that takes money is the least testable part of it. The payment step cannot be exercised safely either, because Rekaz refuses a payment provider's test credentials inside a tenant: <code>Tabby:TestPublicKeyNotAllowed</code> reads <code>The entered public key is for testing only (Test)</code> . Please enter the production public key, or contact Tabby customer support, and <code>Tabby:TestSecretKeyNotAllowed</code> says the same for the secret key. That closes the one workaround a merchant could otherwise use to drive a payment attempt without a real card. ⚠️ It also means every behaviour the documentation does not state can only be learned by performing it in production. Two such rules are still unmeasured in this report for that reason: whether a reservation created through the API takes the cart lock the platform already applies to unpaid items (REK-006), and what a duplicate write produces, since no write endpoint accepts an idempotency key or a client-supplied reference and no duplicate booking was deliberately created on a live tenant to find out. A third is in this report only because it was provoked against live data: a mobile number Rekaz already holds is refused with 403 (REK-046). A merchant discovers rules of this kind by breaking something in front of customers. The commercial position sharpens the point: API access is sold as a paid addon, titled <code>API Access - Premium Addon</code> under <code>Addon:ThirdPartyApi:Upsell:Title</code> , so merchants are paying for an integration surface that has no environment in which it can be integrated safely.
EXPECTED BEHAVIOUR	Issue merchants a sandbox tenant built on the fixtures Rekaz already maintains for its own test accounts, with separate API credentials, seeded catalog data, a payment step that can be driven to success and to failure without a real card, and webhook delivery to a merchant-supplied URL. It must mirror production behaviour exactly, including statuses and error envelopes, or it teaches rules that

do not hold live. Where a payment provider's own test credentials would serve, accept them in the sandbox tenant rather than refusing them as non-production keys.

**ACCEPTANCE
CRITERIA**

A merchant can create, pay for and cancel reservations and subscriptions in the sandbox with no effect on production data, reporting or invoicing, and can run those flows unattended in CI against sandbox credentials.

A RateLimit error code is documented while the limit, the response signal and the retry guidance are not, although the platform already ships a rate-limit refusal in both languages

DOCUMENTED	A <code>RateLimit</code> code appears in the documented error list. No limit value, no response header and no retry guidance is documented anywhere.
ACTUAL	The anchor is in Rekaz's own API documentation: it lists a <code>RateLimit</code> error code, so the API itself asserts that a limit exists. Nothing beside it states what the limit is, which response carries it, or how long a client should wait. No request across the July 2026 probing was answered with HTTP 429. During the concurrency collapse described in REK-043 the requests returned no throttling response at all: they hung past two minutes. ⚠️ Response headers were not systematically inspected, so this states which statuses came back and what the documentation contains, not that no quota header exists anywhere on the API. What the platform adds is that the refusal path is already built, not that the API uses it. Rekaz ships rate-limit copy in both languages under <code>Platform:RateLimitExceeded</code>, where the English reads <code>Rate Limit Exceeded</code> and the Arabic is fuller and already tells the caller to come back later (<code>تم تجاوز الحد المسموح. يرجى المحاولة لاحقًا</code>). In the same generic error family as <code>Error:EntityNotFound</code> and <code>Error:PageNotFound</code> sits <code>Error:TooManyRequests</code> = <code>you have exceeded the maximum number of attempts allowed. please try again later</code>, and a storefront-facing refusal shown to end customers is shipped too, <code>SP:BeautyPass:RateLimitExceeded</code> = <code>Too many verification requests. Please try again later</code>. Elsewhere in the product Rekaz meters and shows the numbers to an ordinary dashboard user, for example <code>Loyalty:Notifications:RateLimit:NextAllowed</code> = <code>Next send available at {0}</code>. ⚠️ That last one is a business-feature quota on an unrelated feature rather than a per-credential HTTP request counter, and it is not evidence that the public API is metered the same way. It establishes only that refusing on a limit, and telling the caller when to come back, are things this product already does somewhere.
EVIDENCE	I raised this as question 7 in our July 2026 review letter and never received an answer. I measured the concurrency behaviour directly against our live tenant while building the integration in July 2026, comparing sequential and concurrent pagination on the same endpoint minutes apart. I read the rate-limit strings from Rekaz's own platform localization tables, English and Arabic, as loaded by the merchant dashboard, and reviewed them on 2026-07-29. I also searched both tables for rate limit, throttle, quota, 429, retry-after, requests per, limit exceeded and cooldown, and for the Arabic <code>الحد الأقصى للطلبات</code>, and found nothing describing a limit on the API itself. A string table can only show what is present, so that search is inconclusive.
IMPACT	Without a documented limit and without a 429, a merchant cannot tell throttling from an outage, and the two demand opposite responses: back off and retry later, or fail fast and alert someone. A client that guesses wrong either keeps hammering an already-strained API or takes its own booking flow offline over a limit it was never near. The documented <code>RateLimit</code> code makes this worse rather than better, because it tells an integrator a limit exists while leaving both its value and its signal undiscoverable, so the only rational design is to assume the tightest limit imaginable and give up throughput that may not need giving up. That an integrator has to guess at all is the part that is hard to justify, because the same platform, in a different feature, already tells an ordinary dashboard user at what time the next send is allowed. Saying not now, come back at this time, is not foreign to this product. It simply is not said to the API caller.
EXPECTED BEHAVIOUR	Document the limits per endpoint or per credential. Return HTTP 429 with a <code>Retry-After</code> header when one is hit, carrying the legacy envelope with <code>error.code</code> populated as <code>RateLimit</code> rather than the null it carried in every legacy-envelope error response observed. Expose remaining allowance and reset time in response headers on every response, using the conventional <code>RateLimit</code> header family, so a client can slow down before it is refused rather than discovering the limit by being cut off.
ACCEPTANCE CRITERIA	Exceeding a documented limit returns 429 with a <code>Retry-After</code> header and a non-null <code>error.code</code> of <code>RateLimit</code>. The limit values and the header names appear in the documentation. Remaining allowance and reset time are readable from response headers before a limit is reached. A request

that exceeds a limit is refused with that response rather than left unanswered until the caller times out.



Performance and access shape

Measured, not estimated.

Response times on the same endpoint vary by more than an order of magnitude, and no latency expectation is published

DOCUMENTED	No performance characteristics are documented: no latency figure per endpoint, no recommended client timeout, no statement of what a slow response means.
ACTUAL	Measured sequentially against the live tenant: <code>/branches</code> 0.8s to 3.3s, <code>/products</code> 1.2s to 10.8s , <code>/subscriptions</code> 1.5s to 3.4s, <code>/reservations</code> (100 rows) around 6s. A single further read of <code>/products</code> on 2026-07-29 answered in 0.63s , so the observed range on that one endpoint is 0.63s to 10.8s, roughly seventeenfold, on an identical request against an identical catalog of 4 products. That fast reading is reported deliberately: any individual call to this endpoint can be quick, which is precisely why a single measurement cannot settle this item in either direction, and precisely why the expectation needs publishing rather than inferring.
EVIDENCE	Repeated sequential measurement against the live tenant, one request at a time, over a working day within the 27 and 28 July 2026 probing window, plus one further read of <code>GET /products</code> on 2026-07-29. We are reporting that last reading, 0.63s, because it falls below the floor we had previously measured and it widens the spread rather than narrowing it. ⚠️ It is one sample and we are not presenting it as a revised floor; it is a single observation that extends the observed range downward. Concurrency was measured separately and degrades far worse (REK-043).
IMPACT	A seventeenfold spread on the same endpoint makes timeout selection guesswork. A timeout short enough to protect a page load fires on healthy requests; one long enough to be safe leaves a customer watching a spinner for ten seconds. <code>GET /products</code> has the worst tail and is the first call a booking flow must make, because nothing documents a price <code>id</code> as stable and no endpoint documents either of the stable handles the API itself returns on every price, <code>immutableId</code> or <code>sku</code>, as an input (REK-051), so a full catalog fetch sits on the critical path of every booking attempt. On the write path the cost is higher again. There are no idempotency keys and no way to ask about a write by request id, so a booking whose response never arrives has to be reconciled by listing reservations and matching on the customer's mobile number. That path does exist and was measured on the live tenant on 2026-07-28, but it is manual, undocumented and inexact: <code>GET /reservations?customerMobile=</code> narrowed 562 rows to 7, and a four-digit fragment of the same number returned those same 7 rows, so the filter matches as a substring rather than exactly and every candidate row must be re-checked digit by digit. A reservation retrieved this way exposes <code>orderId</code> and <code>orderPaymentStatusString</code> and no checkout URL of any kind, so the customer cannot be sent back to payment automatically. Programmatic access is also a commercial line item on this platform rather than a free convenience. Your dashboard presents it as an add-on titled "API Access - Premium Addon", badged "Premium Addon", with its own "API Access Payment" screen, and it names "Sync customers and bookings" and "Build custom integrations" among what is being bought. It also appears in your plan upgrade material as an entitlement, "API access", so on some plans it may be included rather than purchased separately. ⚠️ That is the dashboard's commercial packaging rather than the API surface, and we have not established which of the two forms applies to this tenant. Either way the position is the same: a merchant paying for programmatic access, in whichever form, is being asked to choose a client timeout, size a connection budget and decide what a slow response means, with no figure from you to choose against. Your dashboard already points merchants at an API documentation surface, offered there as a place to "View the available requests and how to use them.", so there is an established home for these numbers.
EXPECTED BEHAVIOUR	Publish expected latency per endpoint (p50, p95, p99) and a recommended client timeout that will not fire on healthy traffic. Separately, investigate the tail on <code>GET /products</code> , which is both the slowest measured endpoint and the one every booking flow is forced to call first.
ACCEPTANCE CRITERIA	Documentation states p50, p95 and p99 per endpoint plus a recommended client timeout. Over 200 sequential requests to each endpoint, spread across a working day, at most 5% exceed that endpoint's published p95 and none exceeds the recommended client timeout.

Concurrent requests collapse without being refused, so the failure is indistinguishable from an outage

DOCUMENTED	Not documented. No concurrency limit is stated, no guidance on parallel requests is given, and no rate limit is published (REK-038).
ACTUAL	Six page requests fired concurrently made <code>GET /subscriptions</code> hang past two minutes , having answered in 1.5s moments earlier when the same pages were fetched sequentially. No 429 and no <code>Retry-After</code> was returned at any point. The requests were not refused, they simply did not answer.
EVIDENCE	Direct comparison of sequential and concurrent pagination against the live tenant, within the 27 and 28 July 2026 probing window. The sequential baseline for that endpoint was 1.5s to 3.4s (REK-042).
IMPACT	Parallelising pagination is the first optimisation any integrator reaches for, and on this API it makes things dramatically worse while producing no signal that explains why. With no 429 and no <code>Retry-After</code> , the observable behaviour is an unresponsive API, so a merchant concludes the service is unreliable and adds retries, which adds exactly the load that caused it. The safe client behaviour is therefore to serialise every request, which compounds the latency in REK-042, and full enumeration is long to begin with because list responses are silently capped at 100 rows: a <code>maxResultCount</code> of 250 returns 100 rows with HTTP 200 and nothing indicating the request was reduced. There is a further consequence a merchant cannot assess from outside and Rekaz can. The hosted checkout a buyer is redirected to after booking is served from <code>platform.rekaz.io</code> , the same host as the public API, and the storefront was observed rendering the same catalog data the API returns. If that customer-facing path shares capacity with the public API, then a parallel fetch in a back-office job is load aimed at the merchant's own revenue path, and nothing in the documentation warns of it. With no sandbox offered (REK-030), the only place a merchant can discover the concurrency limit is production.
EXPECTED BEHAVIOUR	Either serve concurrent requests without collapsing, or publish a concurrency limit per merchant credential and enforce it by refusing the excess immediately with 429 and a <code>Retry-After</code> header. This is not a new mechanism we are asking you to invent. Your own API documentation already lists a <code>RateLimit</code> error code (REK-038), so a refusal of this shape is already in the API's error vocabulary; what is missing is the limit it refers to, the 429 that should carry it, and the <code>Retry-After</code> that should sit beside it. The platform side of your product also already computes the value such a header needs, refusing an action and returning the time remaining before it may be retried, for example "You have exceeded the maximum number of reservations. Please try again after {RemainingTime}". ⚠ That is a platform refusal, not the public API's, and we are not presenting it as evidence that the API pipeline can emit 429 today; the point is only that computing and returning a retry delay is not new work for you. What the public API does under concurrency is neither refusal nor service: it accepts the request and never answers. A request that will not be served promptly must fail fast rather than hang, and two minutes on an endpoint that answers in 1.5s is worse for the caller than an outright refusal. State also whether the public API and the hosted checkout share infrastructure, so a merchant can judge whether a back-office fetch is load aimed at their own revenue path.
ACCEPTANCE CRITERIA	With more concurrent requests than the published limit, every request in excess is refused with 429 and a <code>Retry-After</code> header within two seconds of being received. Requests within the limit are served rather than queued: on <code>GET /subscriptions</code> , the endpoint measured here, none takes longer than the 3.4s upper bound of its own sequential range, and none hangs. The limit and this behaviour are documented.

REK-051 **MEDIUM** GET /products, GET /reservations/slots, POST /reservations/bulk, POST /subscriptions

A price carries both an immutableId and a sku, and neither is documented as an input anywhere, so resolving a price means downloading the whole catalog

DOCUMENTED	<code>immutableId</code> is returned on every price. No endpoint documents it, or any other price identifier, as a lookup key or as an accepted input. The documentation does not state which of a price's identifiers a caller may persist, nor whether <code>id</code> can change.
ACTUAL	Every price object returned by GET /products carries an <code>immutableId</code> and a first-class <code>sku</code> field, the latter sitting alongside <code>stock</code> and <code>package</code> as a top-level key rather than inside <code>extraProperties</code> , which is empty on every price. Across this tenant's 4 products and 21 prices, every <code>pricing[].id</code> differs from its <code>immutableId</code> , and <code>sku</code> is <code>null</code> on all 21, none of them having been set. No GET /prices/{immutableId}, no lookup by <code>sku</code> , and no filter accepting either value appears in the documentation. The only documented route from a stored handle to the current <code>id</code> is to fetch GET /products and scan every product's pricing array.
EVIDENCE	Documentation review plus live catalog inspection on 2026-07-28, re-read on 2026-07-29. Across the tenant's 4 products and 21 prices, every <code>pricing[].id</code> differs from its <code>immutableId</code> , and on 13 of the 21 the two values share no leading prefix. ⚠ Reading those prefixes as date stamps, which would place the live ids in July 2026 and the immutable ids in 2024, is an interpretation of an identifier scheme Rekaz documents nowhere, and it does not hold uniformly: on the other 8 prices the <code>immutableId</code> sits in the same prefix family as the live id. What was measured is that the values differ on every price. The <code>sku</code> field is not something we are proposing you add. It is already in the read model, and your own product already depends on it: your interface strings define a price-level SKU field and describe it as "Unique identifier used to track inventory and service variants", with the same description in Arabic; SKU appears as an accepted bulk-import column; and your accounting connectors refuse to sync without one, returning "Product SKU is missing for an invoice item." and "Some items are missing SKU codes". ⚠ Those last are platform interface strings, not API behaviour, and we did not confirm in the dashboard that the price-level SKU input is available on this tenant's plan. What they establish is that the field is yours and is already used across your product; that it is returned on every price is established independently by the live API. ⚠ On uniqueness, the only SKU uniqueness validation we found stated anywhere is at product level ("SKU already exists"); we found no equivalent stated for the price-level field, so whether a price <code>sku</code> is enforced unique is something only you can confirm. ⚠ We have not submitted a <code>sku</code> or an <code>immutableId</code> in place of a <code>priceId</code> to GET /reservations/slots, POST /reservations/bulk or POST /subscriptions, so whether any of them would accept one undocumented is untested. The absence of any documented input is what this finding rests on.
IMPACT	What was observed is that the two ids differ on every price. ⚠ Inference, not measurement: the reading that fits is that <code>id</code> is reissued when a price is edited in the dashboard, but no price was edited and re-read to confirm it. Rekaz can confirm or refute that from its own change history, and the documentation does not address it either way. On that reading the only safe handles are <code>immutableId</code> and <code>sku</code> , and neither is documented as an input anywhere, which leaves a merchant two options and no third. Either download the entire product catalog to resolve one price, on the critical path of every booking, against the endpoint with the worst measured tail (REK-042). Or persist the <code>id</code> and accept that nothing promises it will hold, which on that same reading produces a booking flow that starts refusing bookings the next time somebody edits a price in the dashboard, with no code change and no warning. ⚠ That second failure would be silent on the merchant's side and invisible on Rekaz's: the price simply no longer exists. What makes this cheap to close is how much of it already exists. A stable per-price identifier is already in your data model, already returned on every price by GET /products, and already named and described in your own interface. Only one direction of travel is missing: nothing accepts it back.
EXPECTED BEHAVIOUR	Accept <code>sku</code> and <code>immutableId</code> anywhere <code>priceId</code> is accepted, including GET /reservations/slots, POST /reservations/bulk and POST /subscriptions. Failing that, provide a documented lookup that resolves either value to the current <code>id</code> in one request, for example GET /products?sku= or GET /prices?sku=. Either way, state in the documentation which of the three identifiers a caller may

persist, whether `id` can change, and whether a price `sku` is enforced unique across the whole tenant or only within a product.

**ACCEPTANCE
CRITERIA**

A price is given a `sku`. Availability can then be queried and a booking created using only that `sku`, or using only an `immutableId` captured at an earlier date, with no call to `GET /products` in between. Both still succeed after that price has been edited in the dashboard, and the documentation states which identifiers are safe to persist.



Documents and money out

Work Rekaz already performs, and already delivers to the buyer, to a notification and to third-party accounting software, but not to the merchant's own system.

Invoices, credit notes and ZATCA documents are produced by the platform and delivered by machine to everyone except the merchant's own system

DOCUMENTED	<code>lastInvoiceCode</code> and <code>lastInvoiceStatus</code> appear in subscription data. No endpoint is documented for retrieving the invoice itself, its line items, its tax breakdown, its credit notes, or its e-invoicing document.
ACTUAL	Rekaz produces all of it. The platform dashboard has an Invoices section (<code>Menu:Invoice</code>) that issues, voids, prints and exports invoices and credit notes (<code>Invoice:IssueInvoice</code> , <code>Order:Print.Invoice</code> , <code>Order:Print.CreditNote</code> , <code>Permission:ExportInvoice</code> , and the plan feature "Full sales report Excel export (orders, transactions, invoices, credit notes)"). The rendered document is already delivered by machine to parties other than the merchant's own system. The buyer is offered it on the hosted checkout success screen as a link labelled "Download Invoice", whose target is <code>/orders/last/invoice?orderId=<order id></code> , with a receipt beside it at <code>/orders/receipt/<order id></code> . Both are relative to the Rekaz origin, in the same shape as the <code>paymentLink</code> this report documents at <code>/orders/pay/<id></code> . Notification templates attach the document automatically (<code>WhatsappNotifications:Template:Media:InvoicePdf</code> , "Invoice PDF file", described in the dashboard as "This notification attaches a file (such as the invoice) to every message. It's added automatically and isn't editable here."), and there is an "Invoice Link" merge tag. Invoices and payments are pushed to third-party accounting software by the Qoyod integration, "Sync your Invoices and payments with Qoyod accounting software", on demand or on a schedule. E-invoicing exists in full as the ZATCA Phase 2 addon, with CSID onboarding, clearance and reporting submissions, ICV, QR generation, credit, debit and prepayment document types, and a nine-state submission status. 🚫 The two document paths above are buyer-session storefront routes keyed on an order id, not merchant API routes under <code>/api/public</code> , and a merchant integration cannot use them in any case: the endpoints reachable with a merchant credential contain no order route, so a merchant holding <code>lastInvoiceCode</code> has no order id to put in them. Through the API an invoice can be identified by its code and its status read, and nothing further.
EVIDENCE	We read Rekaz's own platform localisation tables and hosted checkout component on 2026-07-29. The checkout bundle builds the invoice link as <code>/orders/last/invoice?orderId=<order id></code> and the receipt link as <code>/orders/receipt/<order id></code> , and renders the first as the anchor labelled <code>CustomerPayment:DownloadInvoice</code> , "Download Invoice". The invoice, credit note, accounting and e-invoicing capabilities come from the platform string tables (<code>Invoice:*</code> , <code>CreditNote:*</code> , <code>Integrations:Qoyod:*</code> , <code>Integrations:Zatca</code> , <code>Taxes:Zatca:*</code> , <code>Enum:ZatcaSubmissionStatus:*</code> , <code>Addon:ZatcaPhase2:*</code>). We cross-checked against the endpoints reachable with our merchant credential, re-confirmed 2026-07-28, which contain no invoice route and no order route. ⚠️ The finding is the absence of a documented or reachable merchant endpoint, not the absence of the capability. Invoicing is plan-gated (<code>Platform.InvoicesModule</code> , <code>Platform.InvoicesLimits</code> , <code>PlatformFeature.AllowUnpaidInvoiceIssuance</code>) and ZATCA Phase 2 is a paid addon, so what any one tenant has enabled will vary. That changes who the endpoint would return a document to, not whether it should exist.
IMPACT	Rekaz already hands the rendered document to the buyer, to a WhatsApp notification and to third-party accounting software. The merchant's own system, which is where reconciliation actually happens, gets a code and a status. Anything that needs the document rather than the fact of it therefore routes through a person opening the dashboard and exporting by hand. That step is per invoice and scales with the business: this one tenant held 101 subscriptions and 562 reservations when counted on 2026-07-28. The same applies to the e-invoice a Saudi merchant is asked to produce. Rekaz generates it, submits it and tracks its clearance with ZATCA, and the merchant cannot fetch it back. It also compounds REK-026: a merchant deciding whether to keep Rekaz as the invoicing system of record cannot confirm through the API what Rekaz filed on its behalf.
EXPECTED BEHAVIOUR	Expose over the merchant API what the dashboard, the notification templates and the accounting integration already read. An endpoint that returns an invoice by code, with line items, totals, the tax breakdown, any credit notes raised against it, and a durable link to the same rendered document the

**ACCEPTANCE
CRITERIA**

buyer is offered at "Download Invoice", plus the ZATCA submission status and cleared document where Phase 2 is active for the tenant. Since the storefront already serves that document at `/orders/last/invoice?orderId=<order id>`, the smallest useful version of this is a merchant-authenticated route that resolves `lastInvoiceCode`, or a reservation reference, to the same artefact, since a merchant has no way to obtain an order id. The same endpoint should serve reservation orders as well as subscriptions.

A merchant can call a documented endpoint with the value of `lastInvoiceCode`, or with a reservation reference, and receive the corresponding invoice including line items, totals and the tax breakdown, with no use of the dashboard and without needing an order id it has no way to obtain. The response carries a durable link to the same rendered document served at `/orders/last/invoice?orderId=<order id>` and offered to the buyer as "Download Invoice" on the hosted checkout. Where a credit note exists for that invoice, it is returned with it. Where ZATCA Phase 2 is active for the tenant, the submission status and the cleared document, or a durable link to it, are included.

No refund endpoint, although the dashboard performs the refund in full, including the reversal back to the payment provider

DOCUMENTED	No refund endpoint is documented.
ACTUAL	No refund endpoint appears in the documentation or among the endpoints reachable with a merchant credential. That refunds are dashboard-only is now quotable rather than inferred. The platform dashboard has a Refunds section (Menu:Refunds) that refunds in full or in part, by service line or by transaction (Refunds:ByLineItem , "Services"; Refunds:ByTransaction , "Transactions"), against a coded reason taxonomy (Enum:RefundReasonType:UserRequested , ReservationCancelled , ReservationTimeChanged , ServiceCancelledByProvider , CustomerNotSatisfied , WrongAmountCharged , ServiceQualityIssue , AdminInitiated , Other), with a status model of Pending , Processed and Rejected , refund methods including Cash, Online, Bank Transfer, Point Of Sales and "Tabby or Tamara", and a Refunds report. The online path is a real reversal against the acquirer rather than a bookkeeping entry: the dashboard tracks it through "Refund requested", "Sent to payment provider", "Completed" and "Rejected", and records a "Retrieval Reference Number (RRN)". It raises the accounting document too, since voiding an invoice "voids the invoice and creates a credit note for its value", and credit notes are among the ZATCA document types the platform submits. It performs the downstream state changes as well: "The associated reservation will be cancelled when the refund is processed". None of this is on the API. ⚠️ No refund was attempted: the only tenant available is production (REK-030), where a test refund moves real money.
EVIDENCE	Raised as question 10 in our July 2026 review letter and never answered. Re-confirmed against the documented surface and the endpoints reachable with our merchant credential on 2026-07-28. We read the refund vocabulary quoted above out of Rekaz's own platform localisation tables on 2026-07-29: Menu:Refunds , Refunds:*, Refund:Steps:*, Refund:RRN , RefundMethod:*, Enum:RefundStatus.*, Enum:RefundReasonType:*, Permission:RefundInvoice , Reservation:CancelAndRefund , Invoice:Void:Consequence . ⚠️ That the API has no refund endpoint rests on the documentation and on the enumeration of reachable endpoints, not on those tables, which can show what the platform does and never what it does not.
IMPACT	The asymmetry is visible inside Rekaz's own product. In the dashboard, cancelling a reservation offers the refund in the same step, Reservation:CancelAndRefund , "Cancel & Refund", and on success asks "Reservation cancelled successfully. Would you like to refund the paid amount?". The API exposes that same cancellation, PUT /reservations/{id}/cancel , measured answering 204 on a real reservation, and offers no refund beside it and none elsewhere. A merchant can therefore automate the half of the transaction that takes the room back and none of the half that returns the money. Self-service cancellation cannot be offered on that basis: the customer's money stays put until somebody opens the dashboard, and Rekaz's own refund copy then puts the reversal at "14 working days" on top of that wait. ⚠️ On the one live cancellation, the window reappeared as available immediately afterwards. That is what a released slot looks like, but it does not establish that the Pending reservation was holding the window in the first place, which is undocumented and not determinable from outside (REK-006).
EXPECTED BEHAVIOUR	Expose the refund the dashboard already performs. An endpoint accepting an order, invoice or reservation reference, a full or partial amount, a refund method and a reason drawn from the existing RefundReasonType values, producing the same effects as a dashboard refund: the provider reversal on the online path, the credit note, the cancellation of the associated reservation or subscription, and a webhook event. The platform's existing constraints can carry over unchanged and should be documented rather than left to be discovered: "Online payments can be refunded online only once (partially or fully)."; "Available balance ({0} riyal) is insufficient to process the refund."; and "Payment processor fees are not refunded when the amount is refunded.". Where the refund method is one the platform records rather than disburses, bank transfer being the case the dashboard already flags to the operator, the documentation should say plainly that the API call books the refund and does not move the money. If refunds are deliberately dashboard-only, say so in the documentation and say why.

**ACCEPTANCE
CRITERIA**

A merchant can issue a full or partial refund through the API against a reservation or an invoice, receive the refund's status and, on the online path, its Retrieval Reference Number, observe the order's payment status move to the values the platform already uses for it (`Enum:OrderPaymentStatus.Refunded` , and `Enum:OrderRefundFilterStatus.PartiallyRefunded` for a partial), retrieve the credit note it produced, and receive a webhook event for it. The refund appears in the dashboard's Refunds report indistinguishably from one issued in the dashboard. Or the documentation states plainly that refunds are dashboard-only.



Credentials

Security posture that affects every merchant on the platform, not one.

Rotating a key cuts it off at the moment it is pressed, and the credential a booking integration must hold also reads every customer record

DOCUMENTED	Not addressed. No key rotation procedure is documented.
ACTUAL	A generated key is displayed once and cannot be re-read afterwards, only rotated or replaced. The dashboard's API keys page does hold several named keys side by side, one per integration, so a merchant can create a second key, deploy it and delete the first. What it does not offer is a safe rotation of an existing key: the confirmation on the rotate action states that "The current Base64 value will stop working immediately. Update the integration that uses it.", so the one operation Rekaz labels rotation has no window in which the new value can be verified while the old one still serves traffic. The page also reports nothing about whether a key is still in use, so a merchant following the create-then-delete sequence has no signal for when the old key is safe to remove. Separately, and independently of rotation, the issuance surface offers no choice of scope. The key inspected on 2026-07-28 read the full customer list: <code>GET /customers</code> with it returned 287 records carrying names, mobile numbers and email addresses.
EVIDENCE	Dashboard behaviour at <code>platform.rekaz.io</code> under User Management > API Keys, inspected 2026-07-28, together with Rekaz's own platform localisation table, read 2026-07-29: <code>ApiKeys:Create</code>, "Create API key"; <code>ApiKeys:PageDescription</code>, "Create a separate API key for each integration. Rotate or delete it when needed."; <code>ApiKeys:CreateDescription</code>, "Choose a name that makes the key easy to recognize, such as Website or Accounting."; <code>ApiKeys:NameAlreadyExists</code>; <code>ApiKeys:EmptyTitle</code>, "No API keys yet"; per-key <code>Rename</code>, <code>RotateSecret</code> and <code>Delete</code>; <code>ApiKeys:Base64Description</code>, "This is the only credential value you need to save."; and <code>ApiKeys:RotateConfirmationDescription</code>, "The current Base64 value will stop working immediately. Update the integration that uses it." ⚠️ The rotation behaviour rests on that confirmation text and on the issuance surface, not on an attempted rotation: rotating a live key takes the tenant's booking path down and no sandbox is offered in which to rehearse it (REK-030). ⚠️ We searched the same tables for a per-key last-used timestamp and for any per-key scope restriction and found neither. A string table cannot prove absence, so the accurate statement is that if either exists it is absent from the shipped interface vocabulary and from the documentation. ⚠️ API access is a paid addon ("API Access") and the keys page sits behind a platform feature flag (<code>Platform.ApiKeysManagement</code>), so what a given tenant is shown may differ from the above. The customer-list figure is measured on this tenant, not inferred.
IMPACT	Showing a secret once is good practice. The gap is what happens next. The action Rekaz names "Rotate key" invalidates the current value at the moment it is pressed, so the one operation labelled as rotation is the one that cannot be performed without failing live requests. The safe sequence is create, deploy, delete, and a merchant can carry it out, but nothing on the page reports whether a key is still being used, so the last step is taken on faith: remove the old key too early and booking, availability lookups and back-office views start failing on a credential the merchant believed was idle. What makes this worth fixing rather than tolerating is the scope of what is being rotated. The credential a booking integration is required to hold is also a full read of the merchant's customer base, 287 records on the tenant measured, carrying names, mobile numbers and email addresses. A merchant who suspects that key has leaked is therefore rotating a personal-data credential with no way to see whether the old value is still serving traffic. Scoping is the part that reduces the exposure; the rotation ergonomics decide how fast a merchant can act on it.
EXPECTED BEHAVIOUR	Two things. (1) Make rotation safe on a key's own value, or document create, deploy, delete as the supported procedure and give it the signal it needs. A grace period on the rotate action, in which the previous and the new value are both accepted for a stated window, is the smaller change and matches what the word rotation implies. Either way, show a last-used timestamp per key, so a merchant can confirm the old key is idle before removing it. (2) Offer a narrower scope at issuance, so the credential a booking integration must hold is not also the credential that reads every customer record. A key limited to the catalog, availability and the merchant's own writes would make a leaked or unrotated key a materially smaller exposure than it is today. Rekaz's own front end ships the ABP and OpenIdDict resource strings, including a scope-consent screen, so a scoped-credential

**ACCEPTANCE
CRITERIA**

mechanism appears to be present in the framework already, which would make this configuration of something existing rather than a new subsystem. Document both, including the rotation procedure.

A merchant can move an integration from one credential to another with no request failing, following a procedure Rekaz documents, and can confirm before removing the old credential that it is no longer serving traffic, from a last-used timestamp shown per key. A key issued under the narrower scope can query the catalog, query availability and create a reservation, and is refused with a documented 4xx on `GET /customers` .

Open questions

Sent in our July 2026 review letter. Several findings above exist only because we never got an answer, so we answered some ourselves by testing against production, which is the only environment available (REK-030).

#	QUESTION	RELATED
Q1	Can the hosted payment page be branded with the merchant's identity, or served from a merchant subdomain (for example pay.mazj.org)?	REK-027
Q2	Is there any way to mark an invoice paid from outside Rekaz, so a merchant can collect through their own Saudi gateway while Rekaz remains the invoicing and ZATCA authority?	REK-026
Q3	Is there a sandbox or test tenant, or does all development run against production data?	REK-030
Q4	Are webhook payloads signed? If so, which header and what is the verification procedure? If not, what do you recommend for verifying origin?	REK-005
Q5	Do unpaid Pending reservations expire automatically and release their slot? After how long?	REK-006
Q6	What are the actual rate limits on the public API, and do the public API and the hosted checkout share infrastructure?	REK-038, REK-043
Q7	Is there a refund endpoint, or is refunding dashboard-only?	REK-035
Q8	Is any phone-verification primitive planned, in any form? It is the one change that would let a merchant bind a booking to an account without asserting an identity nobody proved.	REK-046

What we answered ourselves, and what it cost. The original letter asked eleven questions. Three are absent here because we settled them by measuring against production, which is the only environment on offer (Q3): there is no idempotency mechanism on any write, and across 192 real slots the contradictory state in the documented example never occurs. The identity rule behind REK-046 we could only establish by breaking our own site. On 2026-07-28 we shipped a change that always sent the customer details a visitor had typed, rather than binding the booking to an account nobody had verified, and learned from the resulting outage that Rekaz refuses that payload with 403 for any mobile number it already holds. Booking failed for every returning customer until it was withdrawn the same day. That is an expensive way to learn a rule the documentation does not state, and a sandbox would have made it a five-minute test.



What works well

Stated for balance, and because it is true. This report exists because the model underneath is worth extending, not replacing.

- `POST /reservations/bulk` with inline `customerDetails` creates the customer and the reservation in one call, which is a genuinely good design for a first-time buyer. ⚠️ It works only for a mobile number Rekaz has not seen before; for a returning customer the same call is refused (REK-046).
- `GET /customers?mobileNumber=` narrows 287 customer records to 1, with or without the leading plus, and answers quickly.
- `PUT /reservations/{id}/cancel` answers 204, and the cancelled window was observed available again on the next slots request. ⚠️ Whether that is the cancellation releasing a held slot, or whether an unpaid reservation was never holding one, cannot be told apart from outside (REK-006).
- `skipCount` and `maxResultCount` paginate correctly and return disjoint pages.
- The slot model (overlapping windows sliding at one-hour granularity, sized by the price duration) is a good design once understood.
- The catalog, availability, reservation, subscription and package model covers a real coworking business without forcing the merchant to rebuild an operations system. That is the reason this report exists rather than a migration plan.

The one sentence version. Rekaz already does almost everything in this document. The hosted checkout verifies a phone number, applies a coupon and serves an invoice; the dashboard refunds through the acquirer, records a payment taken elsewhere, holds a slot for an unpaid cart and issues several named API keys. A merchant who builds on the Merchant Public API instead reaches none of it, and there is no note anywhere saying so. That is the whole report: the platform is good, the API is a narrower product than the platform, and the difference is invisible until it has already cost you a sale.

We are happy to provide request and response captures for any finding, to re-run any measurement, or to meet if that is faster. Every item above is reproducible against our tenant on request.